



INSTITUTO DE CIÊNCIA E TECNOLOGIA
UNIVERSIDADE ESTADUAL PAULISTA

ENGENHARIA DE CONTROLE E AUTOMAÇÃO

TRABALHO FINAL - ESTRUTURA DE DADOS

Professor:

Prof. Dr. Leopoldo André
Dutra Lusquino Filho

Aluno:

Pedro Seeger Maciel
RA: 251270858

30 de junho de 2026

Sumário

1	Introdução	2
2	Objetivos	2
3	Dataset	2
4	Tratamento de Dados	3
5	Estruturas Trabalhadas	3
5.1	Hash table	4
5.2	Árvore AVL	5
5.3	Lista Ligada	7
5.4	<i>Trie</i>	8
5.5	<i>Union-Find</i> e BFS	9
6	Otimização: <i>Patricia Trie</i>	11
7	Funcionalidades do Programa	11
8	<i>Benchmarks</i> sem restrições	13
8.1	Cenário A	13
8.2	Cenário B	14
8.3	Cenário C	14
9	Complexidade Temporal Teórica e Real	15
9.1	Grupo A: Hash Table, Árvore AVL e Lista Ligada	15
9.2	Grupo B: <i>Trie</i> e <i>Patricia Trie</i>	17
9.3	Grupo C: <i>Union-Find</i> e BFS	18
10	<i>Benchmarks</i> com restrições	18
10.1	Limitação de Memória	18
10.2	Limitação de Processamento	19
10.3	Latência	19
10.4	Dados Corrompidos	19
10.5	Algoritmos Menos Otimizados	20
11	Aplicabilidade das Comparações	20
12	Conclusões	21
13	Uso de IA Generativa	21

1 Introdução

O projeto consiste em utilizar uma base de dados do site de avaliação de filmes "*Rotten Tomatoes*" para realizar diversas operações com nomes de atores e filmes e suas respectivas avaliações, sejam elas da crítica ou do público. A base de dados é dinâmica e altera-se a qualquer momento, a depender das ações efetuadas pelo usuário. Utilizam-se 5 estruturas de dados no programa, sendo elas: hash table, árvore AVL, lista ligada, Trie, e Union-Find. O programa foi programado na linguagem "C++". Estão inclusos no programa *benchmarks* para comparação de tempos de criação e execução entre tipos de estruturas de dados que efetuam as mesmas tarefas e resolvem os mesmos problemas de maneiras diferentes. Além disso, também estão inclusos *benchmarks* com condições restritivas, utilizados para testar como as estruturas se comportam com contratempos como, por exemplo, restrição de memória ou processamento.

2 Objetivos

Os objetivos do trabalho foram:

1. Utilizar 5 estruturas de dados, sendo três da ementa do curso, e duas complementares, externas ao curso;
2. Ser capaz de adicionar, remover e procurar por elementos, além de efetuar 3 operações adicionais;
3. Efetuar *benchmarks*, com e sem restrições, e comparar o desempenho de estruturas de dados que resolvem os mesmos tipos de problemas;
4. Efetuar, em pelo menos uma estrutura, um tipo de otimização, e compará-la com sua versão não-otimizada.

3 Dataset

Um dos requisitos para a escolha do dataset é possuir ao menos 10.000 elementos. O dataset escolhido possui 17.712 filmes, com aproximadamente 204.000 atores diferentes, portanto, cumpre o requisito para ser considerado válido. Entre os motivos da escolha deste dataset específico, encontram-se a utilização do site pelo autor do projeto, além da aplicabilidade real de um programa com esses dados, que torna o processo de procura da filmografia de um autor, ou das avaliações de um filme mais rápido, prático e possível sem conexão à internet.

O dataset foi encontrado no site "Kaggle" e foi feito pelo autor Stefano Leone (stefanoleone992).

4 Tratamento de Dados

A base de dados está em formato .csv, separado por vírgulas (.). Vale ressaltar que, na listagem de autores, eles também estão separados por vírgulas, porém, encontram-se dentro de aspas (Por exemplo: "Ator A, Ator B, Ator C"). Ao tratar os dados para a criação das estruturas, foi necessário criar exceções para os diversos casos de vírgulas localizadas entre aspas.

Alguns filmes possuem informações, como ano de lançamento, ausentes. Nestes casos, utilizamos o valor -1 para tratá-los. Foram encontrados 1.166 filmes sem ano de lançamento, 44 sem notas da crítica, 296 sem notas da audiência, e 352 sem atores. As estatísticas calculadas no programa, como por exemplo, a média das avaliações de todos os filmes em um intervalo de anos especificado pelo usuário, por meio da identificação dos filmes com valores de -1, consegue entregar resultados precisos sem ser afetado pelos filmes com informações que são relevantes para os cálculos ausentes.

5 Estruturas Trabalhadas

As estruturas selecionadas para o projeto foram as seguintes:

- Das três estruturas da ementa do curso:
 1. Hash table;
 2. Árvore AVL;
 3. Lista ligada.
- Das duas estruturas adicionais externas ao curso:
 1. *Trie*;
 2. *Union-Find*.

Antes de continuar, é relevante explicar o que é complexidade temporal, e que notação utiliza-se para falar sobre ela.

A complexidade temporal de um algoritmo consiste do tempo médio para efetuar alguma ação específica. Para denominar essa complexidade, utiliza-se da notação "Big O". Essa notação é um indicador de quanto o algoritmo tende a desacelerar

de acordo com o aumento do número de dados. Por exemplo, um algoritmo com complexidade temporal $O(1)$ não é afetado, ou é muito pouco afetado pela quantidade de dados que ele possui, demorando um tempo médio constante para encontrar ou adicionar dados. Um algoritmo com complexidade $O(n)$, onde "n" é o número de dados no algoritmo, possui uma correlação linear de um para um no tempo de execução de ações. Neste caso, se o número de dados for igual a 10 ($n = 10$), o algoritmo levará, em média, a metade do tempo do que caso o número de dados fosse igual a 20 ($n = 20$). A imagem abaixo (em inglês) apresenta representações gráficas de algumas das mais famosas complexidades temporais em "Big O".

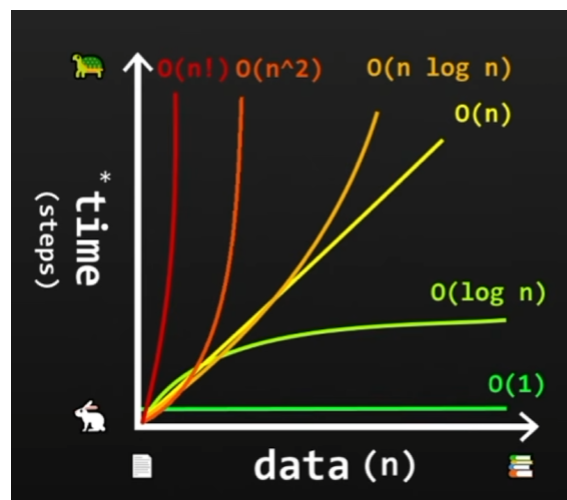


Figura 1: Representação de algumas complexidades temporais. - Youtube: Bro Code

5.1 Hash table

A hash table é uma estrutura que utiliza uma função, denominada *hash function*, e uma chave, decidida a partir do próprio valor do dado, seja ele um número ou uma *string*, para decidir a posição de um dado em sua tabela. Em alguns casos, duas entradas diferentes podem possuir uma mesma chave, o que atribui ao dado uma posição já ocupada anteriormente. Isso é denominado colisão. Existem diversas maneiras de tratamento de colisões em tabelas hash, e a escolhida para este projeto foi o tratamento por listas ligadas, que será explicada em mais detalhes na seção 5.3. A complexidade temporal para tabelas hash é de $O(1)$ tanto para adição, remoção, e procura de elementos. A pior complexidade possível é $O(N)$, caso todos os dados encontrem-se na mesma lista ligada.

A imagem abaixo representa o funcionamento de uma tabela hash com tratamento de colisões por listas ligadas. Observa-se que, no cálculo da posição na tabela do filme

"Avatar", a *hash function* calculou a mesma posição que o filme "Matrix". Portanto, criou-se uma lista ligada, e o filme "Matrix" possui um ponteiro que aponta para o filme "Avatar".

Neste projeto, hash tables foram utilizadas para encontrar um filme pelo nome e para encontrar a filmografia de um ator.

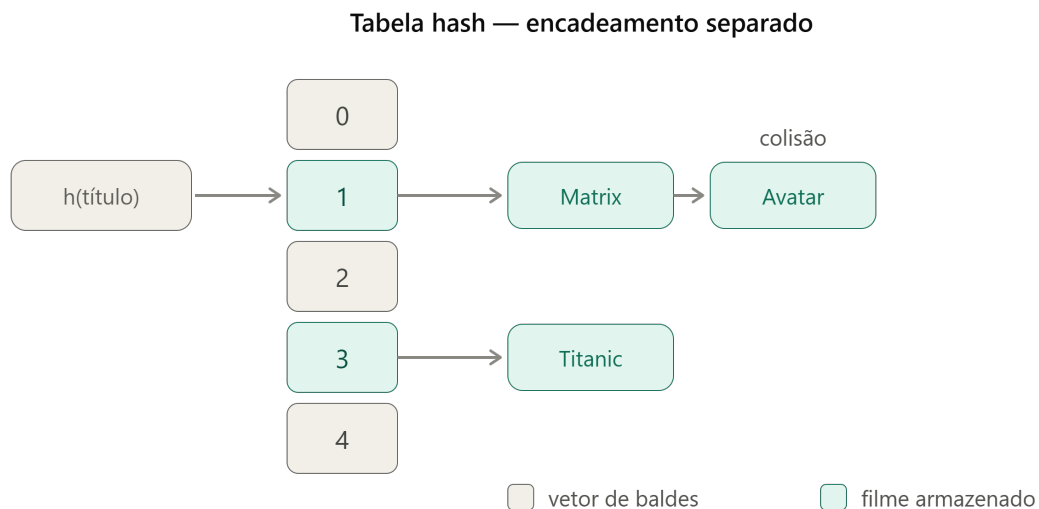


Figura 2: Diagrama de tabela hash com tratamento de colisões por listas ligadas. -
Fonte: Claude

5.2 Árvore AVL

A árvore AVL é uma árvore binária de busca (BST) que se balanceia automaticamente por meio de rotações. A diferença de altura entre dois ramos da árvore só pode tomar os valores -1, 0, ou 1. As rotações servem para impedir que a diferença de altura saia dos valores permitidos. A complexidade temporal de uma árvore AVL é sempre $O(\log(n))$.

Uma árvore binária de busca, se o dado for maior que a raiz, coloca-se à sua esquerda. Se o dado for menor que a raiz, coloca-se à direita. BSTs comuns não possuem mecanismos que detectam assimetria e redistribuem os nós caso as inserções tendam a serem feitas somente, ou em sua maioria, em um só lado. Portanto, dependendo da ordem de inserção dos dados, a árvore pode parecer tornar-se uma lista ligada, tornando sua complexidade temporal cada vez mais parecida com $O(n)$, assim como a citada lista ligada.

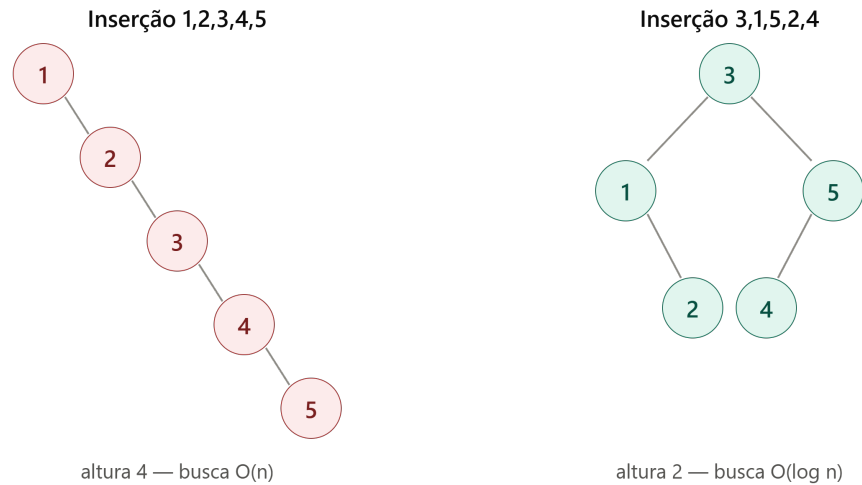


Figura 3: Demonstração de como diferenças na ordem de inserção de dados alteram o balanceamento e complexidade temporal da árvore - Fonte: Claude

Na árvore AVL, rotações em seus ramos permitem que a árvore se auto-balanceie, e assim mantenha a sua complexidade temporal de $O(\log(n))$

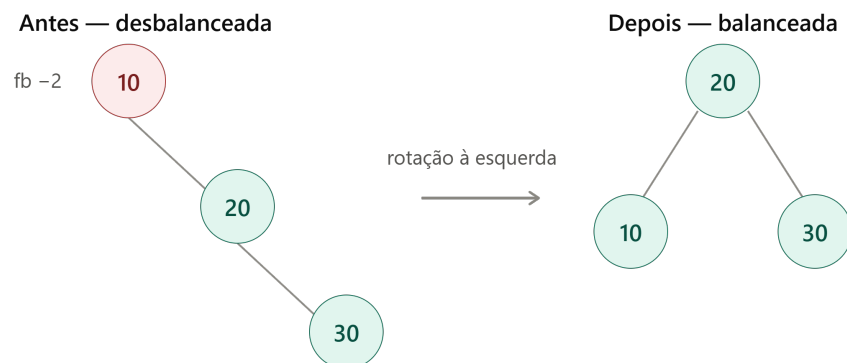


Figura 4: Representação de uma rotação à esquerda em uma árvore AVL. - Fonte: Claude

Quando existem desbalanceamentos LL (esquerda-esquerda) ou RR (direita-direita),

uma rotação para o lado oposto balanceia a árvore. Em casos de "zigue-zague", com desbalanceamentos LR (esquerda-direita) ou RL (direita-esquerda), são necessárias duas rotações. Primeiramente, para a direção da primeira letra, e logo após, uma para o lado oposto.

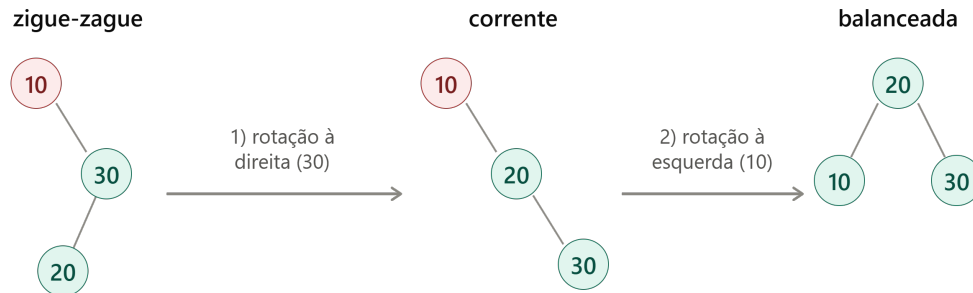


Figura 5: Exemplo de rotação dupla - Fonte: Claude

Neste projeto, árvores AVL foram utilizadas para encontrar um filme pelo título (em *benchmark*), encontrar média das avaliações em um intervalo de anos e para encontrar os filmes mais bem avaliados em um intervalo de anos.

5.3 Lista Ligada

Uma lista ligada consiste de um ponteiro *head* que aponta para o primeiro elemento da lista. Cada elemento armazena o seu valor e um ponteiro. Para adicionar um elemento à lista, faz-se com que o o ponteiro do último elemento (mais distante do *head*) aponte para o novo elemento. Listas ligadas simples possuem complexidade temporal $O(n)$.

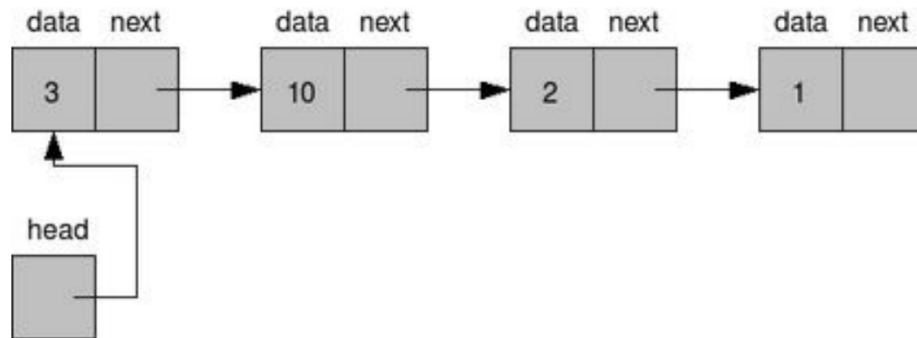


Figura 6: Figura representando uma lista ligada simples. Fonte: Prof. Dr. Leopoldo A. D. L. Filho

Neste projeto, listas ligadas foram utilizadas para encontrar o nome de um filme e suas avaliações (em *benchmark*) e para encontrar a filmografia de um ator.

5.4 *Trie*

Uma *Trie* consiste de utilizar um nó para cada caractere de uma string. Assim como em uma lista, cada nó possui um valor armazenado, porém, a diferença está na quantidade de ramificações diferentes que ele pode ter. A imagem a seguir exemplifica o funcionamento de uma *trie* no seguinte caso:

Imaginemos como armazenar a palavra "test" em uma *trie*. Primeiro, cria-se um nó com a letra "t", e o ponteiro deste nó aponta para a próxima letra, "e". Efetua-se o mesmo processo até chegar na última letra "t". Agora, para adicionarmos outra palavra, "team", reaproveitamos a estrutura até a letra "e", pois seus caminhos (duas primeiras letras) são iguais. Ao caminhar à próxima letra, ocorrem as diferentes ramificações. Agora, será adicionado um novo ponteiro a partir da letra "e" que aponta para um nó com o valor "a", e por fim, este nó com valor "a" aponta para o último nó, com valor "m".

Trie comum — 6 nós

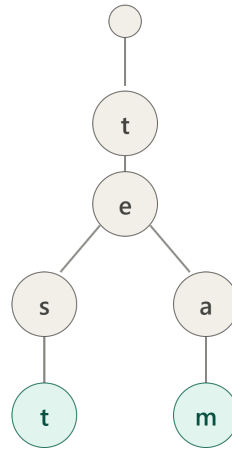


Figura 7: Imagem representando o exemplo do parágrafo anterior. Fonte: Claude

Tries possuem complexidade temporal $O(m)$, sendo "m" o comprimento da chave (*string*) que está sendo adicionada, removida ou inserida. Nesse projeto, *tries* foram utilizadas para encontrar filmes pelo prefixo e para *benchmarks*. Além disso, o tipo de otimização do projeto surgiu a partir da *trie*, onde utilizou-se da *Patricia Trie*, cujo papel e desenvolvimento será explicado na seção dedicada à otimização.

5.5 *Union-Find* e BFS

Estruturas *union-find* foram criadas para resolver o problema: estes elementos estão no mesmo conjunto?

A estrutura tem duas operações: *find*, que sobe a árvore até chegar em sua raiz, e *union*, que une uma raiz à outra. A estrutura sempre coloca a árvore menor abaixo à maior (*union by rank*), e ao encontrar a raiz uma vez, aponta todos os elementos diretamente à ela, o que torna a próxima busca quase instantânea. Portanto, a complexidade temporal em estruturas *union-find* é muito próxima de $O(1)$.

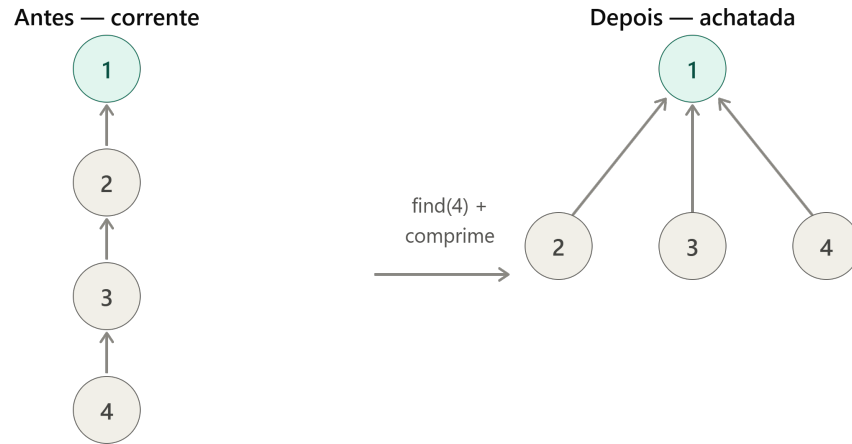


Figura 8: Demonstração de funcionamento de *Union-Find*

Para efeitos de comparação com *union-find*, utilizou-se a estrutura BFS (busca em largura). Essa estrutura permite não só encontrar se dois elementos estão relacionados, mas também o caminho de um até o outro. Por meio de filas, BFS procura primeiramente se algum de seus vizinhos diretos é o elemento "alvo" da procura. Caso não encontre, passa para os vizinhos dos vizinhos, até encontrar o destino ou passar por todos os elementos que estão conectados ao inicial. A complexidade temporal do BFS é $O(V + E)$, onde V é um vértice e E é cada uma de suas arestas.

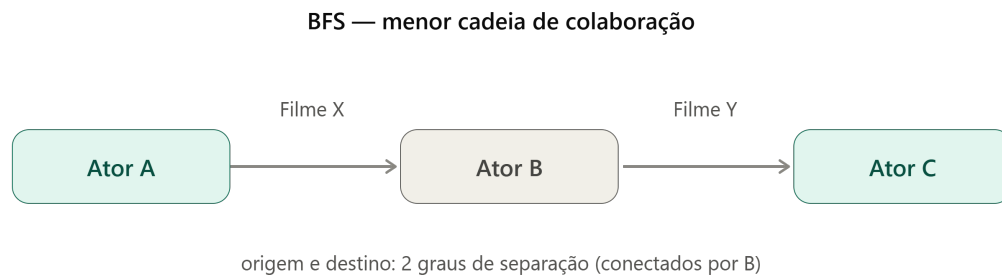


Figura 9: Demonstração de BFS

6 Otimização: *Patricia Trie*

A *Patricia Trie* é uma versão otimizada da *Trie* convencional. No exemplo utilizado para explicar as *tries* convencionais, desenhamos a estrutura para as palavras "test" e "team". Observa-se, enquanto as duas últimas letras das palavras são diferentes, as duas primeiras são iguais, e a *trie* convencional utiliza dois nós para representá-las. A *Patricia Trie* compacta os dois nós "t" e "e" em um nó só, utilizando menos memória. Além disso, é possível compactar as letras "s" e "t" de "test" e as letras "a" e "m" de "team". A figura abaixo apresenta a diferença entre a *Patricia Trie* e a *trie* convencional.

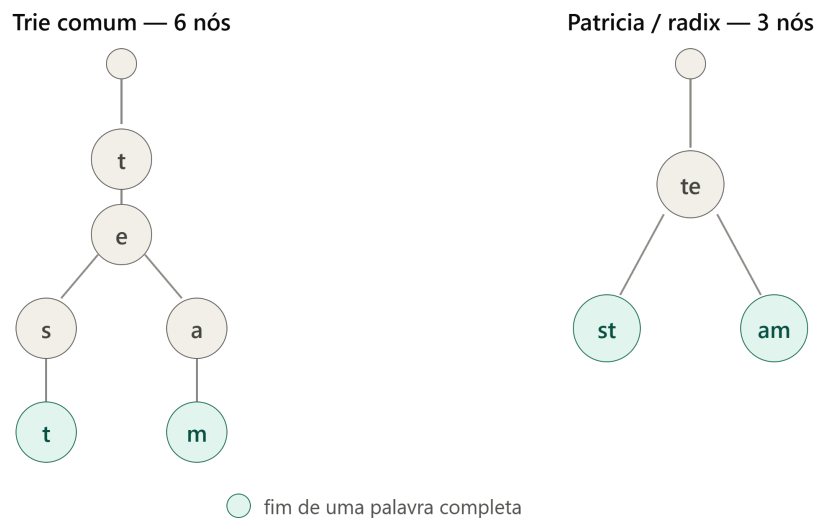


Figura 10: Comparação entre *trie* e *Patricia Trie*. Fonte: Claude

A complexidade temporal da *Patricia Trie* também é $O(m)$, porém, pelas compactações, "m" tende a ser muito menor, assim como será visto na seção de *benchmarks*.

7 Funcionalidades do Programa

O programa possui dez funcionalidades, todas listadas em um menu, onde digita-se o número da função que deseja executar.

```

===== MOVIE DATA SYSTEM =====
1. Find a movie by exact title    (hash table)
2. Search titles by prefix        (Trie)
3. An actor's filmography         (hash + linked list)
4. Rating stats for a year range  (year AVL)
5. Top movies in a year range     (year AVL + sort)
6. Relationship between 2 actors  (3 structures)
7. Add a movie                   (insert)
8. Remove a movie                (remove)
9. Run performance benchmarks    (Groups A/B/C)
10. Run restriction scenarios     (5 categories)
0. Exit

```

Figura 11: Imagem do menu do programa. Fonte: Própria

1. "*Find a movie by exact title*" (encontrar filme pelo título exato);
2. "*Search titles by prefix*" (encontrar filmes pelo prefixo);
3. "*An actor's filmography*" (encontrar a filmografia de um ator);
4. "*Rating stats for a year range*" (estatísticas de avaliações para um intervalo de anos);
5. "*Top movies in a year range*" (melhores filmes em um intervalo de anos);
6. "*Relationship between 2 actors*" (relação entre dois atores);
7. "*Add a movie*" (adicionar um filme);
8. "*Remove a movie*" (remover um filme);
9. "*Run performance benchmarks*" (rodar *benchmarks* de performance);
10. "*Run restriction scenarios*" (rodar cenários de restrição).

8 *Benchmarks* sem restrições

Foram efetuados três benchmarks diferentes, em cenários A, B e C. A máquina que executou os benchmarks possui um processador "Intel Core i5-9400F 2.90GHz", e o sistema operacional é o "Windows 11 Home".

8.1 Cenário A

No cenário A, compararam-se três estruturas diferentes: lista ligada, hash table e árvore AVL na mesma tarefa de encontrar um filme pelo seu nome. Comparou-se o tempo de criação da estrutura com todos os elementos inclusos, e posteriormente, o tempo de procura. Os dados encontrados foram estes a seguir:

Tabela 1: Grupo A: build e busca por título em função de n . Build em ms (total); busca em ns/operação.

n	Build (ms)			Busca (ns/op)		
	Lista	Hash	AVL	Lista	Hash	AVL
1000	0.04	0.24	0.47	1308	41	119
2000	0.08	0.34	0.53	2983	44	140
4000	0.13	0.65	1.17	6791	50	145
8000	0.26	1.56	3.20	14369	69	175
full	0.57	4.82	6.48	19645	90	173

Na tabela abaixo estão localizadas estatísticas mais detalhadas sobre a hash table e suas colisões, assim como requerido para o trabalho:

Tabela 2: Métricas de colisão da tabela hash (encadeamento separado).

Métrica	Valor
Entradas	17105
Buckets	32768
Buckets usados	13327
Fator de carga	0.52
Colisões	3778
Maior cadeia	5

Tabela 3: Grupo A: métricas adicionais (dataset completo).

Métrica	Lista	Hash	AVL
Memória (KB)	276	1389	1400
Consulta aleatória (ns/op)	141622	63	232
Remoção (ns/op)	2752	106	357
Latência combinada (ns/op)	283	77	162

8.2 Cenário B

No cenário B, comparou-se o desempenho da *trie* convencional e da *Patricia Trie*, que é sua otimização. A tarefa a ser efetuada foi a busca de um filme através do seu prefixo.

Tabela 4: Grupo B: Trie comum vs. Patricia (busca por prefixo).

Métrica	Trie comum	Patricia
Build (ms)	35.57	8.27
Nós	185367	24104
Memória (KB)	57927	8466
Busca (ns/consulta)	416865	54927

Observa-se que, na *Patricia Trie*, todas as estatísticas melhoraram drasticamente. Talvez a mais impressionante seja a do número de nós, que diminuiu em aproximadamente 150.000.

8.3 Cenário C

No cenário C, comparou-se o desempenho da *Union-Find* e da BFS. A tarefa foi encontrar se existe uma conexão entre dois atores através de filmes. Por exemplo: Ator 1 e 2 atuaram em filmes diferentes, porém, o ator 3 atuou nos filmes de ambos atores 1 e 2, portanto, há conexão entre atores 1 e 2.

Tabela 5: Grupo C: Union-Find vs. BFS (conectividade entre atores).

Métrica	Union-Find	BFS
Build (ms)	145.33	188.62
Consulta (ns)	60	783274

Vale ressaltar que embora nessa tarefa em específico *union-find* possua um desempenho muito melhor, uma vantagem de usar BFS é que essa estrutura consegue identificar o caminho que liga os dois atores, ou seja, por quais filmes eles estão ligado. *Union-Find* não consegue identificar qual caminho os conecta.

9 Complexidade Temporal Teórica e Real

Utilizando gráficos com as complexidades temporais e os dados obtidos nos benchmarks, conseguimos comparar a complexidade temporal teórica com o tempo de execução real.

9.1 Grupo A: Hash Table, Árvore AVL e Lista Ligada

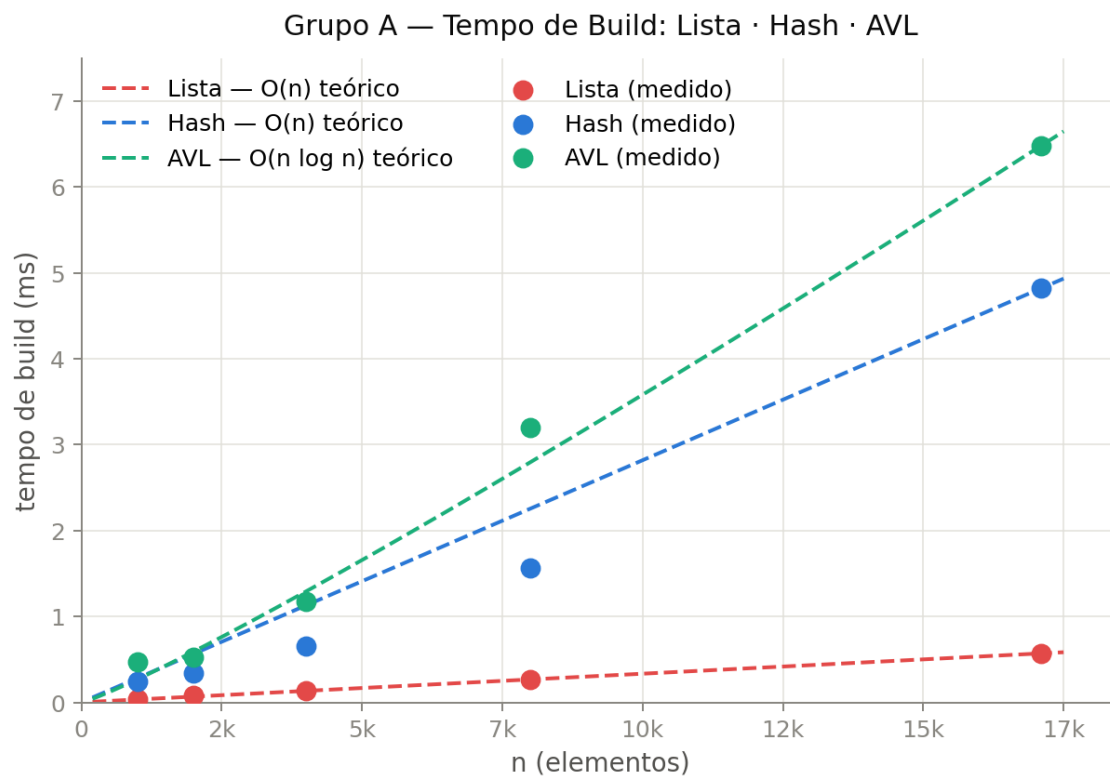


Figura 12: Gráfico comparando o tempo teórico de construção com o tempo encontrado. Fonte: Claude

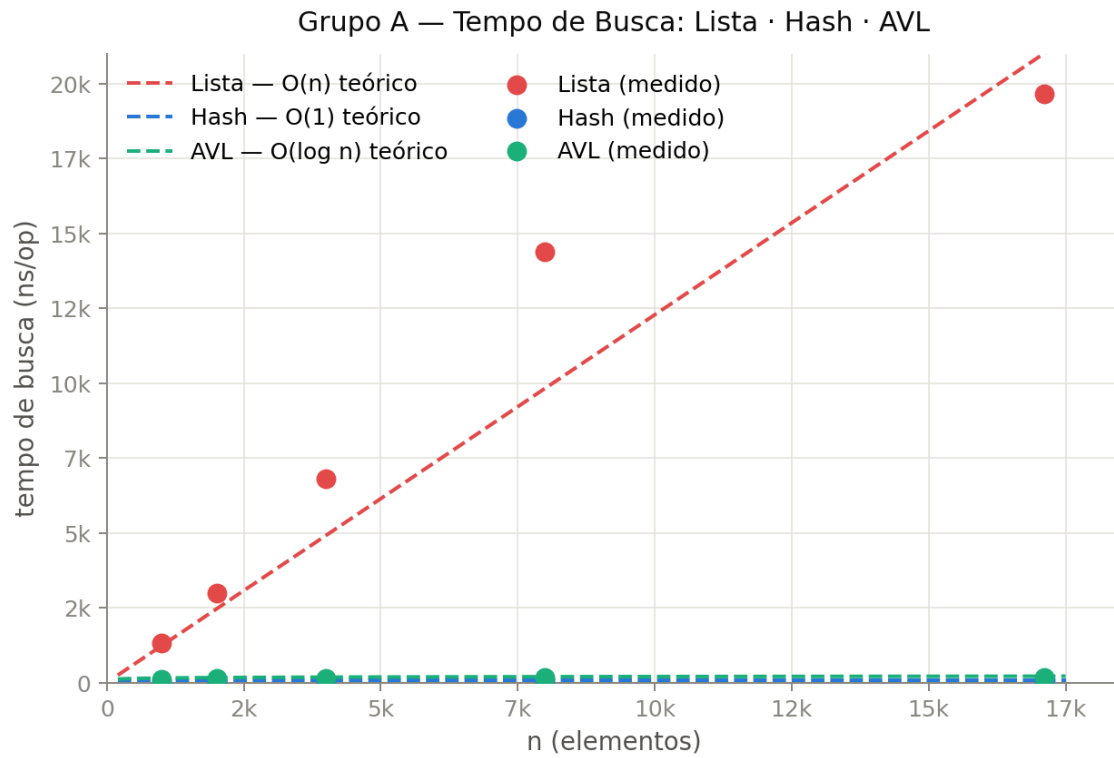


Figura 13: Gráfico comparando o tempo teórico de busca com o tempo encontrado.
Fonte: Claude

Observa-se que, em ambas as figuras, os pontos estão relativamente próximos de suas curvas teóricas esperadas. Quanto maior a quantidade de testes, mais próximas as médias tentem a ser de suas curvas esperadas.

9.2 Grupo B: *Trie* e *Patricia Trie*

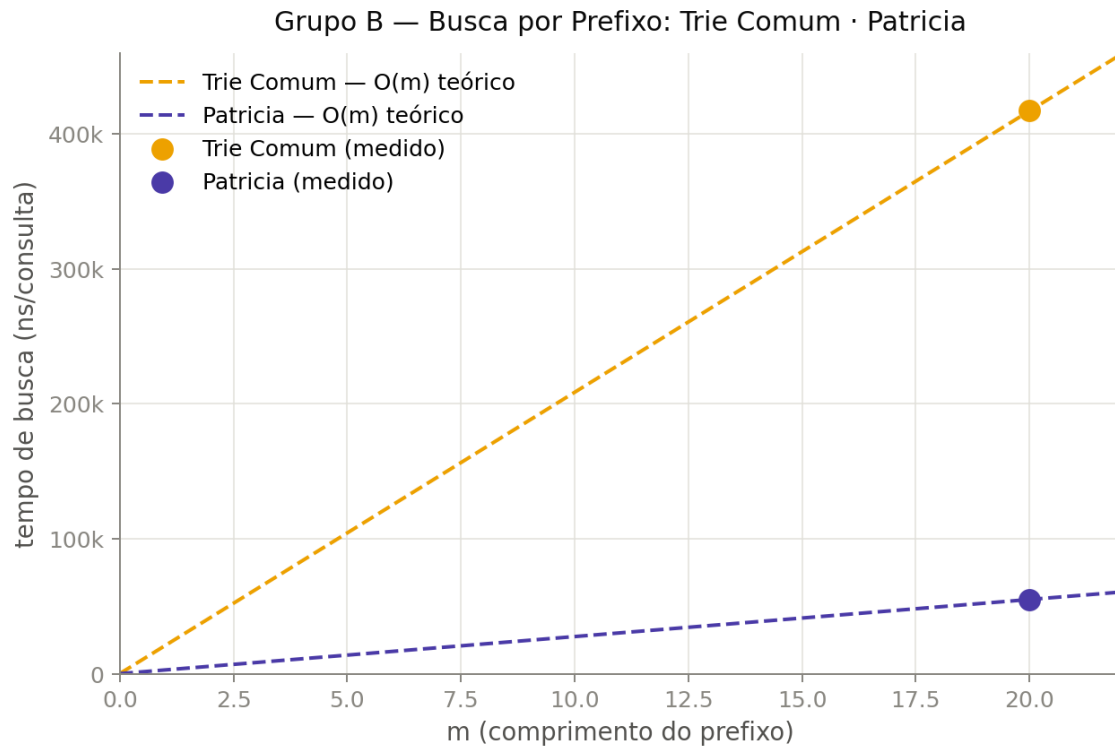


Figura 14: Comparação da curva teórica com os resultados encontrados. Fonte: Claude

È evidente observar como a otimização e agrupamento dos nós por meio da *Patricia Trie* afetou positivamente a velocidade de execução das procuras, muito além do esperado $O(m)$ teórico.

9.3 Grupo C: *Union-Find* e BFS

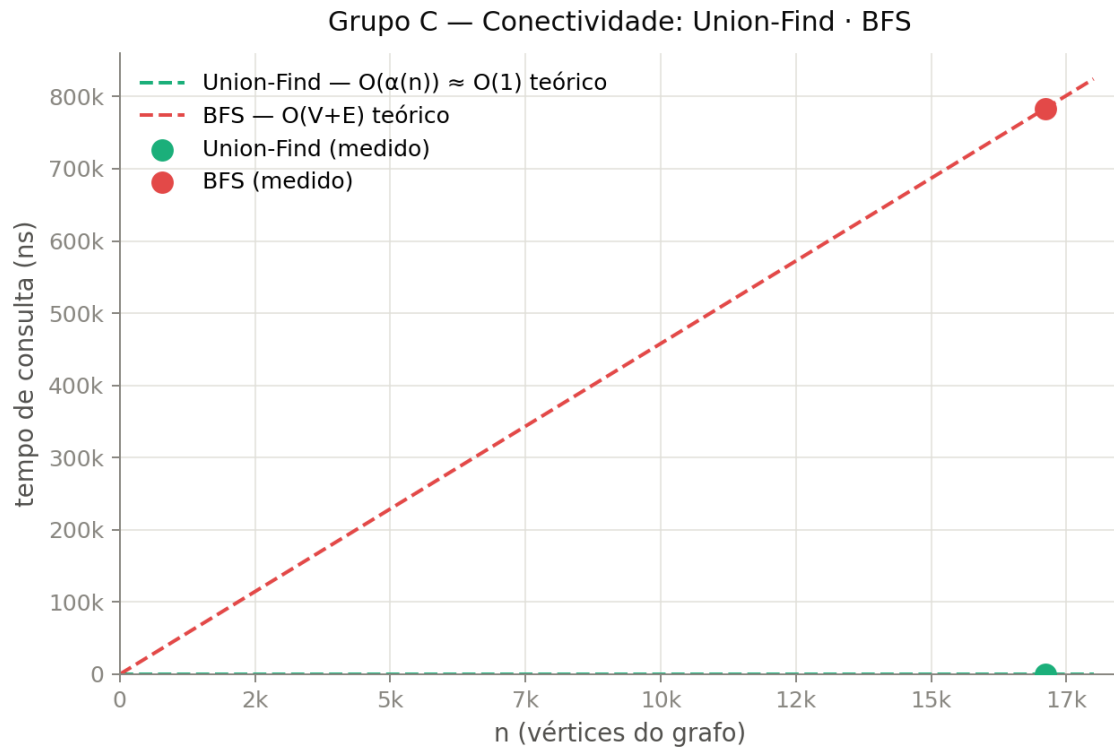


Figura 15: Gráfico comparando o tempo teórico de busca com o tempo encontrado.
Fonte: Claude

O gráfico mostra como o *union-find* é muito mais eficiente do que o BFS para esta ação de encontrar relação entre atores.

10 *Benchmarks* com restrições

Assim como requisitado, foram efetuados cinco *benchmarks com restrições*. E ainda é possível rodar com mais uma restrição, ao utilizar o comando "start /affinity 1" ao executar o programa pelo Prompt de Comando.

10.1 Limitação de Memória

A estrutura utilizada foi a árvore AVL

Tabela 6: Limitação de memória: 500 elementos

	Sem lmitação	Com limitação
Inseridos	17712	511
Rejeitados	0	17201
Altura da árvore	16	11

Com a limitação de memória, não foi possível armazenar todos os elementos na árvore.

10.2 Limitação de Processamento

Foram utilizadas as estruturas hash table e lista.

Tabela 7: Comparação do orçamento de 1000000000ns e de 1000ns

	Violações(1000000000ns)	Violações(1000 ns)
Tabela hash	0	2
Lista encadeada	0	1942

A tabela hash causou muito menos violações nessas condições de restrição

10.3 Latência

Tabela 8: Comparação com e sem latência ($500\mu s$): 200 operações

Sem latência (ms)	Com latência (ms)
0.3337	0.3663

Utilizou-se uma soma em um acumulador para exemplificar que, independente da ação feita, a latência afetará o tempo total de execução.

10.4 Dados Corrompidos

Utilizou-se a árvore AVL dedicada à função 4 e 5 do programa

Tabela 9: Comparação entre arquivo limpo e 20% corrompido

n	Limpo	Selection Sort
n	4304	3468
Média	54.8906	54.8803
Desvio Padrão	27.7834	27.8972

10.5 Algoritmos Menos Otimizados

Neste caso, ao invés de utilizarmos o comando `std::sort`, que possui complexidade temporal $O(n \log(n))$, criou-se um algoritmo *selection sort*, que organiza em complexidade temporal $O(n^2)$.

Tabela 10: Comparação entre `std::sort` e Selection Sort.

n	<code>std::sort</code> (ms)	Selection Sort (ms)
4304	0.1708	9.4687
16508	0.6705	213.222

Com 4304 filmes, o algoritmo mais lento demorou aproximadamente 55 vezes mais tempo para organizar, e com 16508 filmes, aproximadamente 318 vezes.

11 Aplicabilidade das Comparações

As comparações foram feitas corretamente pois apenas comparam estruturas que resolvem o mesmo tipo de problemas. A comparação do Grupo A comparou hash table, árvore AVL e lista ligada para resolver a mesma situação: achar o nome de um filme. Todas as três estruturas tem capacidade para efetuar esta ação.

No Grupo B, comparou-se o desempenho da *Trie* e da *Patricia Trie*. Uma estrutura é uma otimização da outra, portanto, resolvem o mesmo problema, porém com eficiências diferentes

No Grupo C, foi comparado o desempenho de *Union-Find* e BFS na tarefa de encontrar uma relação entre dois atores diferentes. Ambas estruturas conseguem efetuar esta mesma tarefa, portanto, são comparáveis. Vale reforçar que, apesar de BFS ter sido menos eficiente em encontrar se dois elementos são relacionados, possui a vantagem de saber o caminho percorrido do início ao fim.

12 Conclusões

Por fim, conclui-se que a criação de uma aplicação com a utilização de uma base de dados do site de avaliações "*Rotten Tomatoes*" a fim de facilitar a pesquisa de informações de filmes e atores e torná-la acessível sem conexão à internet é possível com a utilização de diversas estruturas de dados. Comparou-se o desempenho de estruturas que resolvem os mesmos problemas, e seus desempenhos com a simulação de diversas limitações. Também avaliaram-se as vantagens e desvantagens da utilização de cada estrutura, o que leva a uma decisão mais informada na escolha da estrutura mais adequada à um projeto.

13 Uso de IA Generativa

O uso de IA generativa limitou-se somente à escrita de códigos no programa. Em acordo com as regras, nenhuma parte do relatório foi escrita com IA generativa. Os logs das conversas tidas para execução do trabalho podem ser encontradas nos links abaixo e no arquivo "README" no projeto submetido ao GitHub

<https://claude.ai/share/8d43d482-878b-4974-ace1-5c018c2b6efe>

<https://claude.ai/share/a80ca4c6-6ea2-4c87-9bff-a5f22df394c1>

<https://claude.ai/share/d585ea3c-d62c-4185-a420-d0fba841c309>